

Referral System & ID Generation

Developer Implementation Brief

Project: User-Centric E-Commerce Platform

Platform: Flutter (Android & iOS)

Prepared by: Rivora Tech Private Limited

Version: 1.2 | Date: March 2026

1. System Overview

This document defines the complete business logic for the Cascading Referral & ID Generation system to be implemented in the Flutter mobile application. It covers ID creation rules, tree population logic, reward distribution, and Flutter implementation guidance.

Key Principle

Users drive platform growth through a system-driven cascading tree. Every new user is automatically placed in the next available slot (left-to-right, breadth-first). No manual placement is allowed.

2. ID Generation Rule

2.1 Core Formula

Every time a user pays their weekly subscription, the system calculates how many new ID slots to create using the following formula:

IDs Created = floor(**Effective Weekly Amount** ÷ **₹3.54**)

Remainder = Effective Weekly Amount – (IDs Created × ₹3.54)

2.2 Carry-Forward Logic

Any remainder that cannot form a full ID slot is NOT discarded. It is retained and added to the next week's subscription amount before the formula is applied again.

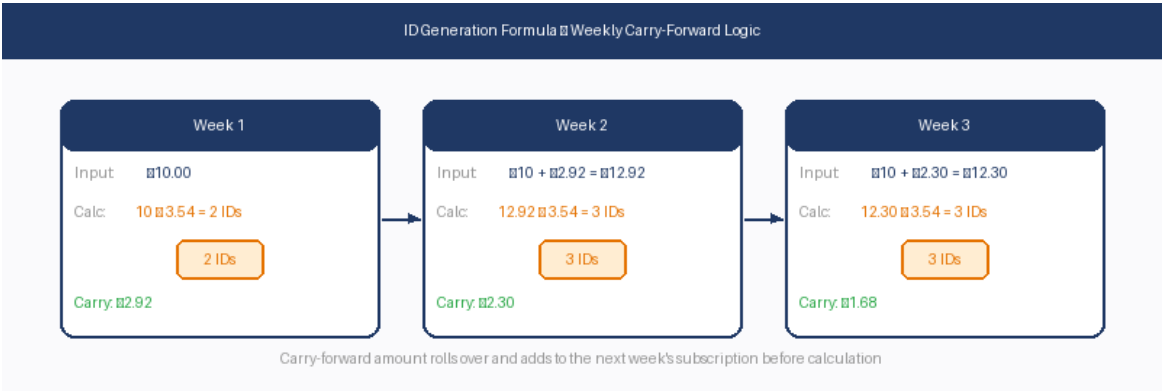


Figure 1 — Weekly ID generation with carry-forward (₹10/week subscription)

2.3 Week-by-Week Example (₹10/week subscription)

Week	Calculation Carry Forward
Week 1	₹10.00 ÷ ₹3.54 = 2 IDs Carry: ₹2.92

Week 2	$\text{₹}12.92 \div \text{₹}3.54 = 3 \text{ IDs} \mid \text{Carry: ₹}2.30$
Week 3	$\text{₹}12.30 \div \text{₹}3.54 = 3 \text{ IDs} \mid \text{Carry: ₹}1.68$
Week 4	$\text{₹}11.68 \div \text{₹}3.54 = 3 \text{ IDs} \mid \text{Carry: ₹}1.06$



Important:

The carry-forward amount must be stored per user in the database and retrieved every week before running the ID generation calculation.

3. Cascading Tree Population

3.1 Population Rule

- Every new user occupies the left-most available empty ID slot in the entire tree.
- Slot assignment is 100% system-driven — no manual placement.
- Upon joining, N new empty ID slots are created downstream of the new user's position (N = IDs from their subscription formula).
- The tree has no depth limit.

3.2 Traversal Order

The system must use Breadth-First Search (BFS) to find the next available slot. This guarantees left-to-right, top-to-bottom filling order across all levels.

BFS Pseudocode for finding next available slot:

```
queue = [root_node]
while queue is not empty:
    node = queue.dequeue()
    if node.is_empty: return node ← place new user here
    else: queue.enqueue(node.children)
```

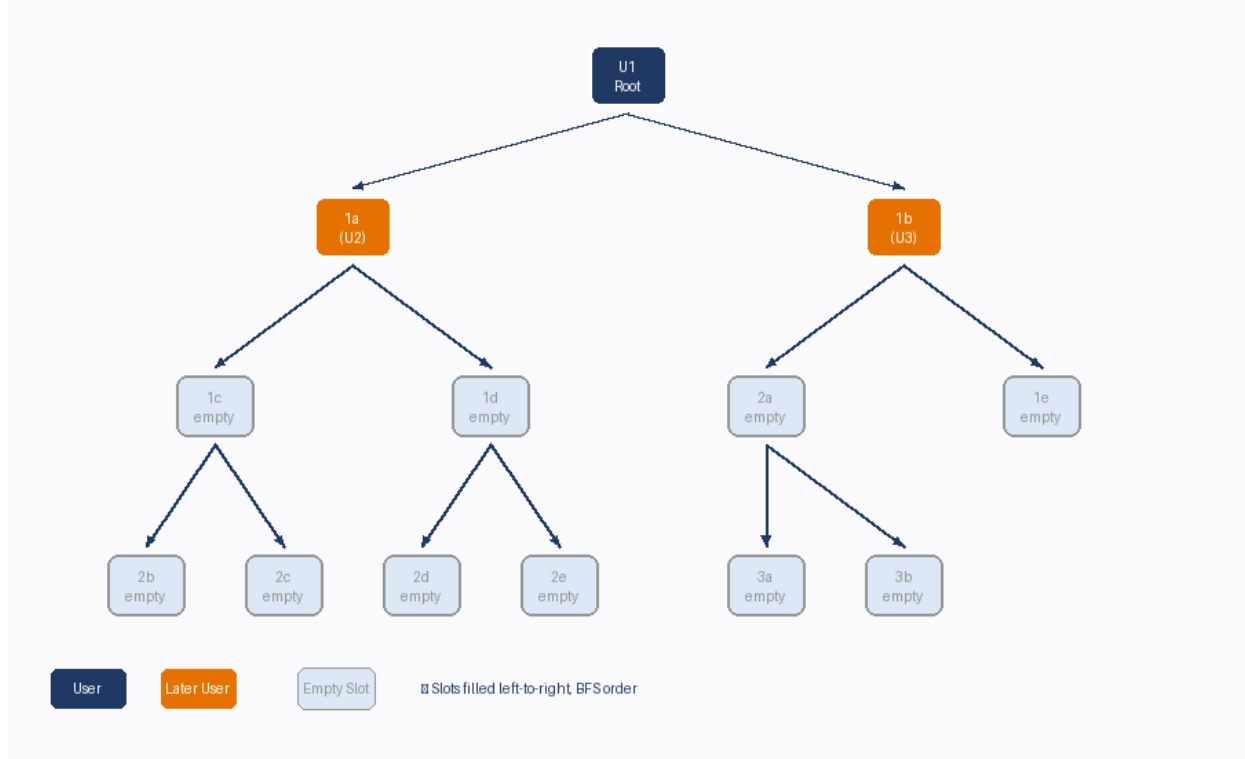


Figure 2 — Cascading tree after 3 users join (₹10/week, 2 IDs per user)

3.3 Step-by-Step Tree Example (₹10/week → 2 IDs per user)

User 1 joins — occupies Root

- Placed at Root position.
- 2 IDs created: Layer 2 → [1a, 1b]

User 2 joins — occupies 1a (leftmost empty)

- BFS finds 1a as first empty slot.
- 2 new IDs created under 1a: Layer 3 → [1c, 1d]

User 3 joins — occupies 1b

- BFS finds 1b as next empty slot.
- 2 new IDs created under 1b: Layer 3 → [2a, 1e]

Tree State After 3 Users:

Layer 1: User 1 (Root)

Layer 2: User 2 (1a) | User 3 (1b)

Layer 3: [1c empty] [1d empty] [2a empty] [1e empty]

4. Vendor Commission & ID Generation

Vendor-generated IDs follow the same ₹3.54 per ID formula and the same BFS left-to-right tree population rules. However, there is one critical difference:

Property	Vendor ID Generation
Formula	$\text{floor}(\text{Commission Amount} \div ₹3.54)$
Carry-forward	Yes — same logic as users
Tree Population	Same BFS left-to-right rules
Reward triggered?	NO — vendor IDs do not trigger upstream rewards
Vendor own reward?	NO — vendors have no referral reward structure

Commission Calculation Example

- Grocery Vendor | Weekly Sales: ₹1,000 | Commission: 2% = ₹20
- IDs generated: $\text{floor}(20 \div 3.54) = 5$ IDs | Carry: ₹2.30
- These 5 IDs are inserted into the same unified referral tree via BFS — but NO reward is distributed.

5. Reward Distribution

5.1 Reward Rules

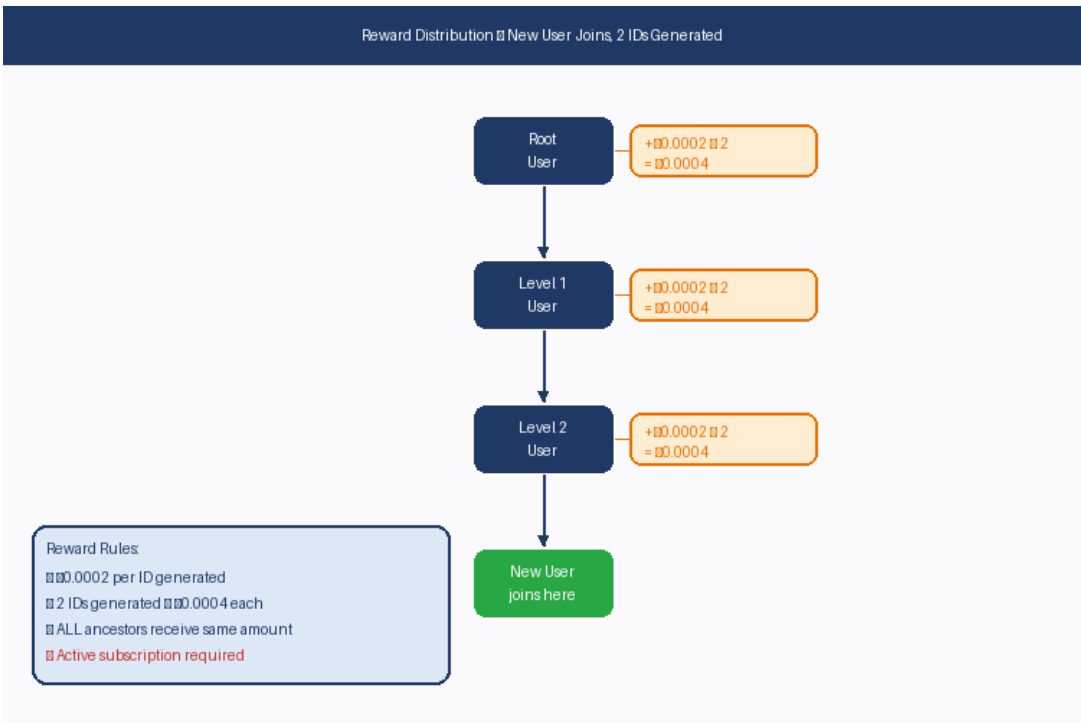


Figure 3 — Reward propagation: every ancestor receives ₹0.0002 per ID generated

Rule	Detail
Trigger	New user ID slot is generated (from user subscription only)
Reward per ID	₹0.0002
Who receives it	Every ancestor in the chain from placed node up to root
Amount per ancestor	₹0.0002 × number of IDs generated (flat, not split)
Eligibility condition	Ancestor must have an active weekly subscription
Inactive ancestor	Their reward is skipped; chain continues upward
Vendor-generated IDs	No reward triggered
Payout frequency	Weekly batch processing

5.2 Reward Calculation Example

Scenario: New user joins and generates 2 IDs. The referral chain has 4 ancestors (L3 → L2 → L1 → Root). L2 has an inactive subscription.

Ancestor	Reward Outcome
Root (L1)	Active → receives ₹0.0002 × 2 = ₹0.0004 ✓
Level 2 User	Active → receives ₹0.0004 ✓
Level 3 User	INACTIVE → reward skipped ✗
Level 4 User	Active → receives ₹0.0004 ✓

Note: The chain does not break at an inactive ancestor.

Rewards continue flowing upward past inactive users — only that specific user's credit is skipped.

6. Wallet, Subscription & Withdrawal

6.1 Wallet

- Each user has an in-app digital wallet.
- Wallet is topped up via UPI Gateway (upigateway.com).
- Wallet balance is used for: weekly subscription payments, product purchases.
- Rewards are credited directly to the wallet.

6.2 Withdrawal Rules

Rule	Value
Minimum withdrawal	₹100

TDS deduction	5%
Admin charge	15%
Service charge	5%
Total deducted	25%
User receives	75% of requested withdrawal amount

6.3 Reward Eligibility Gate

A user receives rewards only if both conditions are true:

- Active weekly subscription is maintained, AND
- The user or any user in their downline has made at least one purchase that week.

7. Flutter Implementation Notes

7.1 Database Schema (Recommended)

users table

id	UUID	PRIMARY KEY
name	TEXT	
subscription_status	ENUM(active, inactive)	
weekly_sub_amount	DECIMAL(10,2)	
carry_forward_amt	DECIMAL(10,4)	-- remainder from last week
wallet_balance	DECIMAL(12,4)	
created_at	TIMESTAMP	

tree_nodes table

node_id	UUID	PRIMARY KEY
user_id	UUID	REFERENCES users(id) -- NULL if empty slot
parent_id	UUID	REFERENCES tree_nodes(node_id)
position	ENUM(left, right, ...)	-- child position under parent
bfs_order	BIGINT	-- pre-computed BFS sequence number
is_occupied	BOOLEAN	DEFAULT FALSE
created_at	TIMESTAMP	

id_generation_log table

id	UUID	PRIMARY KEY
user_id	UUID	-- user who triggered generation
week_start	DATE	
ids_generated	INT	
carry_forward	DECIMAL(10,4)	
source	ENUM(user_subscription, vendor_commission)	

rewards table

id	UUID	PRIMARY KEY
recipient_id	UUID	REFERENCES users(id)
trigger_event	UUID	-- references id_generation_log
amount	DECIMAL(10,4)	
status	ENUM(pending, credited, skipped)	
week_start	DATE	

7.2 BFS Slot Finder — Dart Pseudocode

Use the `bfs_order` column to efficiently find the next available slot without traversing the full tree in-memory:

```
// Efficient DB query approach:  
SELECT * FROM tree_nodes  
WHERE is_occupied = FALSE  
ORDER BY bfs_order ASC  
LIMIT 1;
```

```
// Or in-memory BFS in Dart:  
TreeNode findNextSlot(TreeNode root) {  
    Queue<TreeNode> queue = Queue();
```



```

queue.add(root);
while (queue.isNotEmpty) {
  TreeNode node = queue.removeFirst();
  if (!node.isOccupied) return node;
  queue.addAll(node.children);
}
throw Exception('No empty slot found');
}

```

7.3 ID Generation Service — Dart Pseudocode

```

void generateIDs(User user, double weeklySubAmount) {
  double effective = weeklySubAmount + user.carryForwardAmt;
  int idsToCreate = (effective / 3.54).floor();
  double newCarry = effective - (idsToCreate * 3.54);

  for (int i = 0; i < idsToCreate; i++) {
    TreeNode slot = findNextSlot(root);
    TreeNode newNode = createEmptyNode(parentId: slot.id);
    distributeRewards(newNode, user); // skip for vendor IDs
  }
  user.carryForwardAmt = newCarry;
  user.save();
}

```

7.4 Reward Distribution Service — Dart Pseudocode

```

void distributeRewards(TreeNode placedNode, User triggerUser) {
  // Only for user-subscription-triggered IDs, not vendor
  double rewardPerAncestor = 0.0002;
  TreeNode? current = placedNode.parent;

  while (current != null) {
    if (current.user?.isSubscriptionActive == true) {
      creditWallet(current.user!, rewardPerAncestor);
    }
    // continue even if inactive – chain does not break
    current = current.parent;
  }
}

```

7.5 Weekly Cron Jobs Required

Job	Description
Subscription renewal check	Every Monday — mark users active/inactive based on payment
ID generation run	After subscription payment confirmed — calculate IDs + carry-forward
Reward payout batch	Weekly — process all pending reward credits to wallets
Vendor commission calc	Weekly — compute commissions, generate

	vendor IDs (no rewards)
Carry-forward reset	Do NOT reset — accumulate across weeks until used

7.6 Edge Cases to Handle

- User subscription lapses mid-week: freeze ID generation until next payment received.
- Carry-forward must persist across subscription lapses — do not reset on lapse.
- If tree has no empty slots (shouldn't happen but guard): throw error and alert admin.
- Vendor commission remainder: store per vendor, same carry-forward logic as users.
- Concurrent joins: use DB transactions with row-level locking on tree_nodes to prevent two users claiming the same slot.

8. Quick Reference Summary

Parameter	Value
ID cost	₹3.54 per ID
ID formula	$\text{floor}(\text{effective amount} \div 3.54)$
Carry-forward	Remainder retained, added to next week
Tree fill order	Left-to-right, BFS (breadth-first)
Placement	100% system-driven, no manual override
Reward trigger	User subscription ID generation only
Reward amount	₹0.0002 per ID generated
Reward recipients	All ancestors from placed node to root
Reward per ancestor	$\text{₹0.0002} \times \text{IDs generated}$ (flat, not split)
Inactive ancestor	Skipped — chain continues upward
Vendor IDs	Same formula + BFS, but NO rewards
Withdrawal minimum	₹100
Withdrawal deduction	25% (5% TDS + 15% Admin + 5% Service)
Weekly payout	Rewards processed in weekly batch

— End of Document —